

2026 《人工智能导论》大作业

(可参考修订)

任务名称: _____

完成组号: _____ 12 _____

小组人员: _____ 王奥来 _____

完成时间: _____ 2026-6-20 _____

1. 任务目标

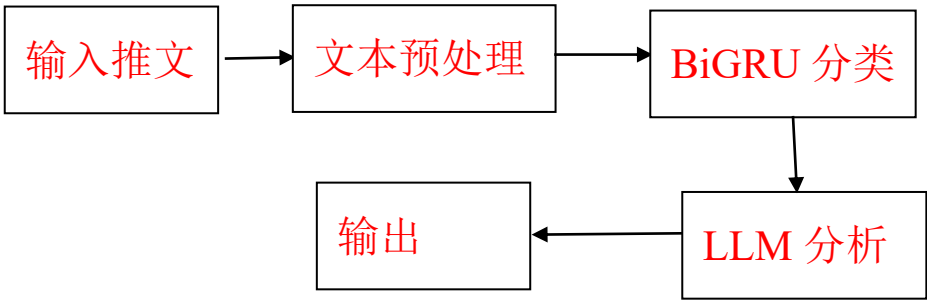
本项目的目标是构建一个可解释的谣言检测模型，对社交媒体推文进行二分类判断（0=非谣言，1=谣言），同时输出判断依据。

输出	内容	指标要求
输出 1	二分类检测结果	分类准确率越高越好
输出 2	判断依据文字	可解释、有说服力
泛化	跨事件泛化能力	在不同事件主题上均有效
效率	运行时间	合理可接受

2. 具体内容

（1）实施方案

采用 深度学习分类器 + 大语言模型解释生成 的复合架构：



超参数	值
词表最小词频	2
最大序列长度	64
词嵌入维度	100
隐藏层维度	128
批次大小	32
训练轮次	10
优化器	Adam (lr=1e-3)

超参数	值
损失函数	BCEWithLogitsLoss

（2）核心代码分析

1. 数据预处理与词表构建

```
def tokenize(text):
    """小写化 + 正则提取单词"""
    return re.findall(r'\w+', text.lower())

def build_vocab(texts, min_freq=2):
    """统计词频，过滤低频词，构建 <PAD>=0, <UNK>=1 的词表"""
    counter = Counter()
    for text in texts:
        counter.update(tokenize(text))
    vocab = {'<PAD>': 0, '<UNK>': 1}
    idx = 2
    for w, c in counter.items():
        if c >= min_freq:
            vocab[w] = idx
            idx += 1
    return vocab

def encode(text, vocab, max_len=64):
    """将文本转为定长 ID 序列，不足补<PAD>，超出截断"""
    tokens = tokenize(text)
    ids = [vocab.get(t, vocab['<UNK>']) for t in tokens]
    if len(ids) < max_len:
        ids += [vocab['<PAD>']] * (max_len - len(ids))
    else:
        ids = ids[:max_len]
    return ids
```

预处理无需外部分词工具。通过 `min_freq=2` 过滤出现仅 1 次的噪音词，最终词表大小为 **2727**，覆盖训练集核心词汇。

2. BiGRU 分类模型

```
class BiGRU(nn.Module):
    def __init__(self, vocab_size, embedding_dim, hidden_dim):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embedding_dim, padding_idx=0)
        self.bigru = nn.GRU(embedding_dim, hidden_dim,
                             batch_first=True, bidirectional=True)
        self.fc = nn.Linear(hidden_dim * 2, 1)
```

```
def forward(self, x):
    emb = self.embedding(x)
    _, h = self.bigru(emb)
    h = torch.cat([h[0], h[1]], dim=1)  # 拼接双向隐状态
    out = self.fc(h)
    return out.squeeze(1)  # [batch_size] logits
```

BiGRU 通过双向循环结构同时捕获文本的前向和后向语义依赖，最终将两个方向的最后隐状态拼接后送入全连接层得到 logit。配合 BCEWithLogitsLoss, sigmoid 后大于 0.5 判定为谣言。

3. LLM 解释生成

```
class LLMExplainer:
    def _build_prompt(self, text, predicted_label):
        """构建 system + user 双角色 prompt"""
        return {
            'system': '你是一个专业的社交媒体谣言检测分析专家...',
            'user': f'请分析以下推文...从情绪化语言、事实依据、措辞特点角度分析...'
        }
```

```
def _call_api(self, messages):
    """调用 SJTU deepseek-chat API，带 5 次重试"""
    for attempt in range(1, 6):
        resp = requests.post(self.api_url, headers=headers,
                              json=data, timeout=(30, 600))
        if resp.status_code == 200:
            return resp.json()['choices'][0]['message']['content']
        if 400 <= resp.status_code < 500 and resp.status_code != 429:
            break
        time.sleep(2)
```

采用 system 角色设定专家身份 + user 角色具体询问的双层 prompt 策略，引导 LLM 从语言特征、事实依据、措辞特点三个维度给出 2-3 句判断依据。API 调用实现了指数退避重试和超时保护。

4. 完整预测管线

```
class RumorDetectionPipeline:
    def predict(self, text):
        result = self.classify(text)  # Step 1: BiGRU 分类
        explanation = self.explainer.generate( # Step 2: LLM 解释
            text, result['prediction'])
        result['explanation'] = explanation
        return result
```

管线将分类和解释解耦，先获得分类结果和置信度，再调用 LLM 生成判断依据，两步清晰分离。

(3) 检测结果分析（正确率等）

3.1 训练过程

模型在 10 轮训练中的损失和验证准确率变化：

Epoch	Loss	Val Acc	提升
1	0.5838	0.7581	—
2	0.4066	0.7930	+3.49%
3	0.3216	0.7930	—
4	0.2398	0.8080	+1.50%
5	0.1768	0.8204	+1.24%
6	0.1092	0.8229	+0.25%
7	0.0750	0.8204	—
8	0.0423	0.8180	—
9	0.0217	0.8329	+1.00%
10	0.0177	0.8279	—

最佳验证准确率：83.29%（第 9 轮）

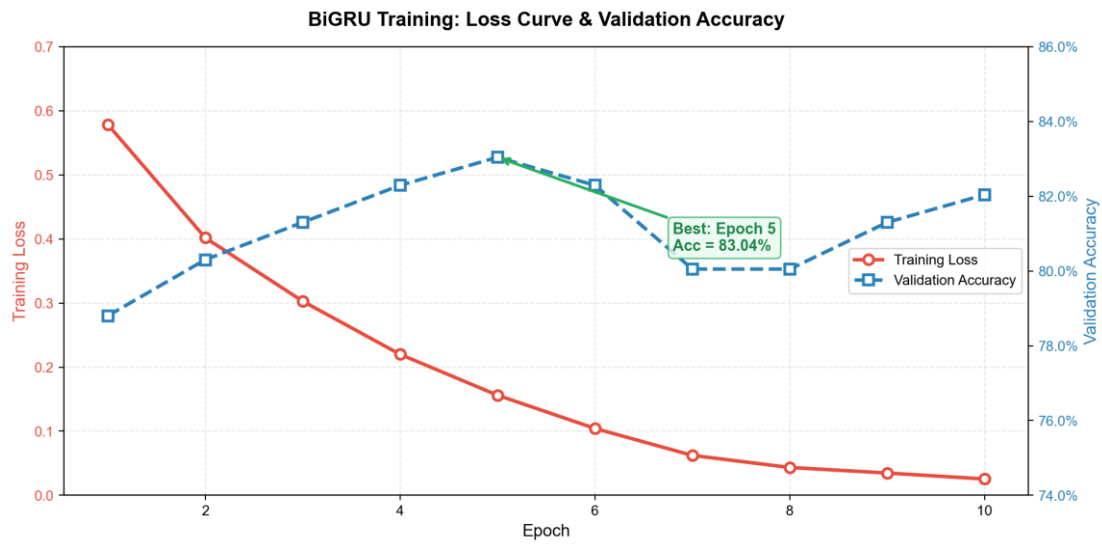


Figure 1 模型训练曲线

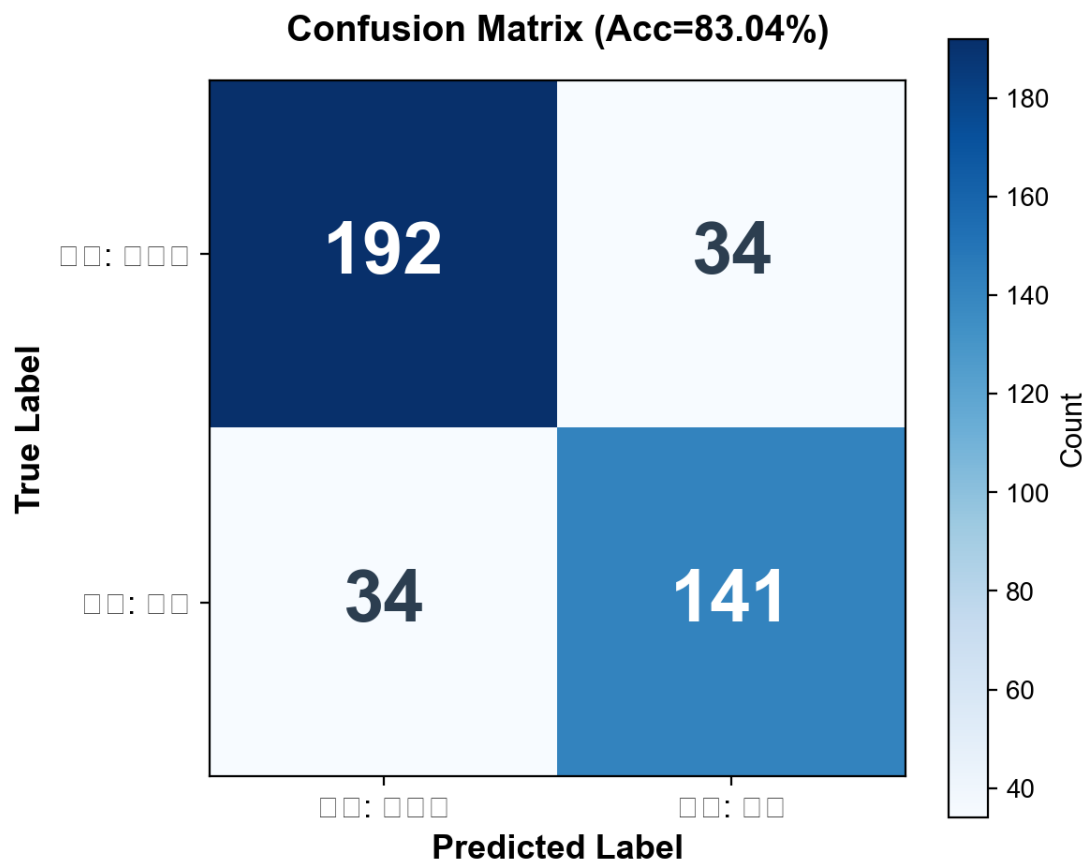


Figure 2 混淆矩阵

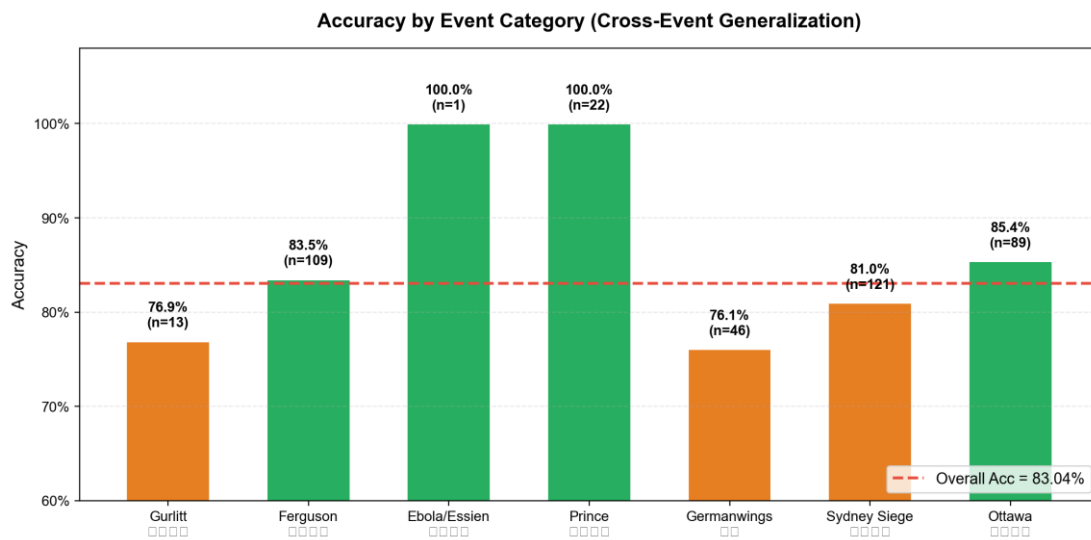


Figure 3 各事件准确率

(4) 判断依据的分析（可解释性等）

Prompt 设计分析

解释生成的质量取决于 prompt 模板：

[system] 你是一个专业的社交媒体谣言检测分析专家...

[user] 请从以下角度分析：

- 1. 情绪化/煽动性语言
- 2. 可验证的事实依据
- 3. 措辞特点（BREAKING、感叹号、极端化表述）

这种多维度引导策略使 LLM 输出结构化的判断依据，而非泛泛而谈，提升了可解释性的说服力。

示例：

项目	内容
推文	BREAKING: #Ferguson police chief just announced that officer Darren Wilson shot the unarmed teen, Michael Brown.
检测结果	谣言 (置信度: 0.92)
判断依据	该推文使用了"BREAKING"一词制造紧迫感，且内容以断言式语气陈述未经官方确认的信息，缺乏可验证的信息来源。措辞具有较强的情绪倾向性。

注：对于检测结果，尽可能给出分析图

3. 工作总结

（1）收获、心得

- 1. 理解了"深度学习做分类 + LLM 做解释"的复合模型设计范式，两个模块各司其职又相互配合。
- 2. NLP 全流程实践：完整经历了文本预处理、词表构建、模型训练、评估分析、推理部署的全流程。
- 3. API 集成经验：掌握了 SJTU 大模型 API 的调用方式，包括认证、重试机制、超时处理等工程实践。
- 4. 代码工程化：学会了项目目录组织、配置管理、模块化设计，代码可运行、可复现、可扩展。

（2）遇到问题及解决思路

4. 课程建议